SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY - INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Informatics

Formal Verification of Earley's Parsing Algorithm, Including Runtime Analysis

Alexander Haberl

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY - INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Informatics

Formal Verification of Earley's Parsing Algorithm, Including Runtime Analysis

Formale Verifikation von Earleys Parser Algorithmus mit Laufzeitanalyse

Author: Alexander Haberl
Examiner: Prof. Tobias Nipkow, Ph.D.
Supervisor: Prof. Tobias Nipkow, Ph.D.
Date: 11.05.2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 11.05.2026 Alexander Haberl

Abstract

TODO Abstract

Zusammenfassung

TODO Abstract Übersetzung

Contents

Abstract iv

Zusammenfassung v

1 Introduction 1

2 Background 2
- 2.1 Context Free Grammars . . . 2
- 2.2 Parse Trees . . . 3

3 Earley Recognizer Formalization 4
- 3.1 Inductive definition . . . 6
- 3.2 Towards executability . . . 7
- 3.3 Workset closure . . . 8
- 3.4 Epsilon treatment . . . 9

4 Executable List based Version 11

5 Efficient Itemlist Datastructure for running time guarantees 12

6 Runingtime Analysis 13

7 Space Analysis 14

8 Expansion to Parse Trees 15

9 Runingtime Analysis of Expanded Algorithm 16

10 Try out examples 17

Bibliography 18

1 Introduction

This is a test sentence that has been changed incredibly much This is another test

2 Background

2.1 Context Free Grammars

Languages are sets of words, which are strings made up of terminal symbols. The whole set of terminal symbols is called the alphabet Σ. One way to define what words are in a Language are context free grammars (CFGs) which introduce new non-terminal symbols and productions that formalize how a string of non-terminal and terminal symbols can be changed to derive words step by step, starting from a single non-terminal. A production expresses one such possible step, where you replace one non-terminal symbol by a string of non-terminal and terminal symbols. The Language of a CFG is then the set of strings containing only terminal symbols that can be reached from the starting non-terminal, following the modifications described by the productions. Strings of non-terminals and terminals are called sententials and strings of only terminal symbols are called sentences.

Formally in Isabelle strings are represented by lists and so to be able to represent sententials non-terminals and terminals are grouped together into the wrapper symbol datatype *sym* which can represent either a terminal or non-terminal symbol.

datatype $('n,\ 't)\ sym = \mathsf{Nt}\ 'n \mid \mathsf{Tm}\ 't$

Productions (A, α) are a pair of a non-terminal A and a sentential α, where the non-terminal is called the left-hand side (lhs) and the sentential is called the right-hand side (rhs) of the production. In text we will write productions like this $(A \to \alpha)$. A set of productions P then defines the so called derivation relation $\mathsf{P} \vdash u \Rightarrow v$, which is formally defined as follows:

$\mathsf{derive} :: ('n \times ('n,\ 't)\ sym\ list)\ set \Rightarrow ('n,\ 't)\ sym\ list \Rightarrow ('n,\ 't)\ sym\ list \Rightarrow bool$
$(\mathsf{P} \vdash u \Rightarrow v) = (\exists A\ \beta\ x\ y.\ u = x\ @\ [\mathsf{Nt}\ A]\ @\ y \wedge v = x\ @\ \beta\ @\ y \wedge (A,\ \beta) \in \mathsf{P})$

This represents one modification step as described informally above. Context free grammars are made up of a set of productions P and a non-terminal as a starting symbol, commonly named S. The Language of a CFG is then defined in terms of the transitive closure of the derivation relation $\mathsf{P} \vdash u \Rightarrow* v$. Specifically by which sentences can be reached from the starting non-terminal $[\mathsf{Nt}\ S]$.

datatype $('n,\ 't)\ Cfg = \mathsf{Cfg}\ (('n \times ('n,\ 't)\ sym\ list)\ set)\ 'n$

$\mathsf{Lang} :: ('n \times ('n,\ 't)\ sym\ list)\ set \Rightarrow 'n \Rightarrow 't\ list\ set$
$\mathsf{Lang}\ P\ A = \{w \mid P \vdash [\mathsf{Nt}\ A] \Rightarrow*\ \mathsf{map}\ \mathsf{Tm}\ w\}$

$\mathsf{LangS} :: ('n,\ 't)\ Cfg \Rightarrow 't\ list\ set$
$\mathsf{LangS} \equiv \lambda G.\ \mathsf{Lang}\ (\mathsf{Prods}\ G)\ (\mathsf{Start}\ G)$

A CFG implicitly defines the sets of terminals and non-terminals by which appear in its productions.

$\mathsf{Nts_syms} :: ('n,\ 't)\ sym\ list \Rightarrow 'n\ set$
$\mathsf{Nts_syms}\ w = \{A \mid \mathsf{Nt}\ A \in \mathsf{set}\ w\}$
$\mathsf{Nts} :: ('n \times ('n,\ 't)\ sym\ list)\ set \Rightarrow 'n\ set$
$\mathsf{Nts}\ P = (\bigcup_{(A,\ w) \in P} \{A\} \cup \mathsf{Nts_syms}\ w)$

$\mathsf{Tms_syms} :: ('n,\ 't)\ sym\ list \Rightarrow 't\ set$
$\mathsf{Tms_syms}\ w = \{a \mid \mathsf{Tm}\ a \in \mathsf{set}\ w\}$
$\mathsf{Tms} :: ('n \times ('n,\ 't)\ sym\ list)\ set \Rightarrow 't\ set$
$\mathsf{Tms}\ P = (\bigcup_{(A,\ w) \in P} \mathsf{Tms_syms}\ w)$

There exist multiple interesting computational questions concerning languages. One very natural question is, given a word w, is it in the Language given by a CFG? This is called recognition and a natural continuation is asking about the derivation of such a word if it is in the Language, which is called parsing.

2.2 Parse Trees

- CFG in Isabelle
- Parse trees in Isabelle

3 Earley Recognizer Formalization

In this thesis we formalize a recognizer first proposed by Earley [1]. We fix for the following chapter a word w and a CFG C which induces the Language L. Earley's recognizer builds sets of items for each index of w, where an item in one of these sets represents that a sub-string of w has been recognized. These items consist of a production, and two natural numbers that are called the dot and from.

datatype $('n,'a)$ *item* = *Item* (*prod*: $('n,'a)$ *prod*) (*dot* : *nat*) (*from* : *nat*)

We will write an item *Item* p d f like this $(A \to \alpha\bullet\beta, f)$, where the production is $(A, \alpha\beta)$ and $|\alpha| = d$. The idea behind the recognizer is that an item *Item* p k l in set $\mathcal{S}_i$ symbolizes that we have recognized the sub-string of w starting at index l and ending at index i and that it can be derived from the partial right-hand side of p that is left of the dot at position k. We call this property soundness.

Definition 1. *Item* p k $l \in \mathcal{S}_i \implies P \vdash$ *slice* 0 k (*rhs* p) $\Rightarrow*$ *slice* l i w.

slice i k s returns the sub-string starting at index i and ending at index k-1. It is equivalent to *take* k (*drop* i s).

For better readability we define some helper functions:

is_complete :: $('n, 'a)$ *item* $\Rightarrow$ *bool*
is_complete $x = (|$*rhs* (*prod* x)$| \leq$ *dot* $x)$

next_symbol :: $('n, 'a)$ *item* $\Rightarrow$ $('n, 'a)$ *sym option*
next_symbol $x =$
(*if* *is_complete* x *then* *None* *else* *Some* (*rhs* (*prod* x) ! *dot* x))

next_sym_Nt $\equiv \lambda x\ A.$ *next_symbol* $x =$ *Some* (*Nt* A)

mv_dot :: $('n, 'a)$ *item* $\Rightarrow$ $('n, 'a)$ *item*
mv_dot $x \equiv$ *Item* (*prod* x) (*dot* $x + 1$) (*from* x)

is_complete checks whether the dot is at the end of the production, *next_symbol* returns an option of the symbol after the dot, *next_sym_Nt* checks if the next symbol is a specific non-terminal and *mv_dot* moves the dot over the next symbol.
We also define a predicate to check if an item is well-formed in the context of the recognizer algorithm, so what items could appear in a set $\mathcal{S}_k$. We check that the production is part of the grammar, that the dot is a valid index into the right-hand side of the production, that the from value is less than or equal to k and that k is less than or equal to the length of w.

wf_item :: $('n, 'a)$ *item* $\Rightarrow$ *nat* $\Rightarrow$ *bool*
wf_item x k = (*prod* $x \in P \wedge$ *dot* $x \leq |$*rhs* (*prod* x)$| \wedge$ *from* $x \leq k \wedge k \leq |w|$)

Earley then goes on to define three operations for the recognizer algorithm which generate items for these sets in an inductive manner. These are called *predict*, *complete* and*scan*. If we have an item in set $\mathcal{S}_i$ with its dot in front of a non-terminal A we predict. So we start recognizing a new a sub-string starting at index i of w from the non-terminal A.

$$\textsc{Predict:}\ \frac{x \in \mathcal{S}_i \qquad \mathit{next_sym_Nt}\ x\ A \qquad p \in P \qquad \mathit{lhs}\ p = A}{\mathit{Item}\ p\ 0\ i \in \mathcal{S}_i}$$

Completion occurs when we have fully recognized an item $B \to \alpha\bullet$, f, meaning its dot is at the end of the right-hand side of the production. In this case we have recognized a sub-string of w starting at f. This recognition started with the non-terminal on the productions left-hand side B. So we can extend the recognition of sub-strings up to f ($P \vdash u \Rightarrow\!* \ \mathit{slice}\ k\ f\ w$) with this sub-string recognition as follows $P \vdash u\ @\ [\mathit{Nt}\ B] \Rightarrow\!* \ \mathit{slice}\ k\ i\ w$. In terms of items this means we are taking an item recognizing a sub-string up to f from set $\mathcal{S}_f$ and can extend its recognition if the symbol after the dot is the non-terminal B.

$$\textsc{Complete:}\ \frac{\begin{array}{ccc} x \in \mathcal{S}_i & \mathit{is_complete}\ x & \mathit{from}\ x = f \\ y \in \mathcal{S}_f & \mathit{next_sym_Nt}\ y\ (\mathit{lhs}\ (\mathit{prod}\ x)) & \end{array}}{\mathit{mv_dot}\ y \in \mathcal{S}_i}$$

The last operation scan occurs when the next symbol of an item is a terminal. In this case we can advance recognition of the sub-string if the next symbol in w is said terminal. In this case we add these items to the next set $\mathcal{S}_{i+1}$, since the length of the recognized sub-string of w has increased by one.

$$\textsc{Scan:}\ \frac{x \in \mathcal{S}_j \qquad j < |w| \qquad \mathit{next_symbol}\ x = \mathit{Some}\ (w\ !\ j)}{\mathit{mv_dot}\ x \in \mathcal{S}_{(j\ +\ 1)}}$$

Lastly we need a starting point for this inductive definition, which is given by the prediction operation on the starting symbol.

$$\textsc{Init:}\ \frac{p \in P \qquad \mathit{lhs}\ p = S}{\mathit{Item}\ p\ 0\ 0 \in \mathcal{S}_0}$$

From these inductive definitions we can derive the following functions producing sets resulting from the operations on a single item.

$\mathit{Predict}\ it\ i = \{\mathit{Item}\ p\ 0\ i \mid p \in P \wedge \mathit{next_sym_Nt}\ it\ (\mathit{lhs}\ p)\}$
$\mathit{Complete}\ \mathcal{S}_k\ B = \{\mathit{mv_dot}\ it \mid it \in \mathcal{S}_k \wedge \mathit{next_sym_Nt}\ it\ B\}$
$\mathit{Scan}\ \mathcal{S}_i\ i = \{\mathit{mv_dot}\ it \mid it \in \mathcal{S}_i \wedge \mathit{next_symbol}\ it = \mathit{Some}\ (w\ !\ i)\}$
$\mathit{Init} = \{\mathit{Item}\ p\ 0\ 0 \mid p \in P \wedge \mathit{lhs}\ p = S\}$

We also define a top level predicate for w called recognized, which returns true if we find a complete item with the start-symbol as its left-hand side in set $\mathcal{S}_{|w|}$. By soundness this means that $P \vdash [\mathit{Nt}\ S] \Rightarrow\!* \ w$ holds.

Definition 2. $\mathit{accepted} = (\exists\, x{\in}\mathcal{S}_{|w|}.\ \mathit{is_final}\ x)$
where $\mathit{is_final}\ x = (\mathit{lhs}\ (\mathit{prod}\ x) = S \wedge \mathit{from}\ x = 0 \wedge \mathit{is_complete}\ x)$

We give an example to better understand and visualize the algorithm: Given a CFG with productions $P = \{A \to AB,\ A \to BA,\ A \to a,\ B \to b\}$ and starting non-terminal A we get the following sets for the word "bab".
TODO maybe make different visualization showing what different operations add

$\mathcal{S}_0$		$\mathcal{S}_1$		$\mathcal{S}_2$		$\mathcal{S}_3$	
$A \to \bullet AB$	0	$B \to b\bullet$	0	$A \to a\bullet$	1	$B \to b\bullet$	2
$A \to \bullet BA$	0	$A \to B\bullet A$	0	$A \to BA\bullet$	0	$A \to AB\bullet$	1
$A \to \bullet a$	0	$A \to \bullet AB$	1	$A \to A\bullet B$	1	$A \to AB\bullet$	0
$B \to \bullet b$	0	$A \to \bullet BA$	1	$A \to A\bullet B$	0	$A \to BA\bullet$	0
		$A \to \bullet a$	1	$B \to \bullet b$	2	$A \to A\bullet B$	1
		$B \to \bullet b$	1			$A \to A\bullet B$	0
						$B \to \bullet b$	3

Table 3.1: Example of the sets constructed by the Earley parser

3.1 Inductive definition

Towards a formal proof of this recognizer, we define one inductive Earley-set, which will be the union of all $\mathcal{S}_i$ where $i < |w|$. To differentiate which set the items are in we will save pairs of Earley-items and a natural number indicating which set they are from. It is constructed inductively by the three operations and the initial set discussed previously.

$\mathit{Earley} :: (('n,\ 'a)\ \mathit{item} \times \mathit{nat})\ \mathit{set}$

For correctness we need to prove the two directions of soundness and completeness. Which entails that if the recognizer accepts a word, it is in the language and the opposite direction, that if a word is in the language then the recognizer accepts it. Soundness is quite easy to prove for this definition, we only need to prove soundness of every item in this set, which can be done by proving that all operations preserve soundness, and the initial set is sound. From these individual proofs overall soundness follows:

Theorem 1. $\mathit{accepted} \Longrightarrow P \vdash [\mathit{Nt}\ S] \Rightarrow* \mathit{w}$

To prove completeness we show that if there is an item in the Earley-set and the rest of the items right-hand side after the dot can derive a sub-string of w following the sub-sting already derived by the item, then the complete item will also be in the set.

Lemma 1. $[\![j \leq k;\ k \leq |\mathit{w}|;\ x = \mathit{Item}\ (A,\ \gamma)\ d\ i;\ (x,\ j) \in \mathit{Earley};\ P \vdash \beta\ x \Rightarrow(n)\ \mathit{slice}\ j\ k\ \mathit{w}]\!] \Longrightarrow (\mathit{Item}\ (A,\ \gamma)\ |\gamma|\ i,\ k) \in \mathit{Earley}$

This is proven using total induction on the length of the derivation and an induction on the length of the rest of the right-hand side of the item. From this the full completeness follows rather easily, as the initial set contains all possible incomplete items after one step of derivation. Therefore if the entire word is derivable one of these items will be complete in the final Earley-set $\mathcal{S}_{|w|}$ and thus the word is accepted by the recognizer.

Theorem 2. $[\![x \in \mathcal{S}_i;\ \mathit{is_complete}\ x;\ \mathit{from}\ x = f;\ y \in \mathcal{S}_f;\ \mathit{next_sym_Nt}\ y\ (\mathit{lhs}\ (\mathit{prod}\ x))]\!]$ $\Longrightarrow \mathit{mv_dot}\ y \in \mathcal{S}_i$

3.2 Towards executability

While this theoretical Earley recognizer is well suited for proves it lends itself badly for execution, because of the inductive definition. Instead the iterative set computation suggested by Earley is better suited. Looking at the four operations of the Earley algorithm we see that only *predict* and *complete* add items to the current set, while *init* is the staring set and *scan* adds item to the next set, resulting in the starting set of the next set. Every operation also only depends on items in sets with lower or equal index, letting us compute the sets in order. So we can split the algorithm into two steps first compute the starting set for set $\mathcal{S}_i$ using *init* or *scan* and then compute the closure of this set under *predict* and *complete*. This can then be repeated until we have computed all sets.

Going forward we will call item sets *bins*, to which we extend the well formedness predicate. The algorithm will follow this general form:

$\mathit{bins} :: \mathit{nat} \Rightarrow ('n,\ 'a)\ \mathit{item\ set\ list}$

$\mathit{bins}\ 0 = [\mathit{Close}\ []\ \mathit{Init}]$

$\mathit{bins}\ (k + 1) = (\mathit{let}\ Bs = \mathit{bins}\ k\ \mathit{in}\ Bs\ @\ [\mathit{Close}\ Bs\ (\mathit{Scan}\ (\mathit{last}\ Bs)\ k)])$

It computes a list of bins as described before, using *Close* to compute the closure. This way we can refine the closure algorithm step by step, while the underlying structure of the algorithm does not change. From now on Bs will always refer to the list of bins computed previously.

To prove that this new algorithm computes the same sets as the fully inductive definition we first use an inductive set closure. It takes a starting set B - either a result of *init* or *scan* - and the previously computed list of bins Bs and returns the closure C under *predict* and *complete*.

$\mathit{Close} :: ('n,\ 'a)\ \mathit{item\ set\ list} \Rightarrow ('n,\ 'a)\ \mathit{item\ set} \Rightarrow ('n,\ 'a)\ \mathit{item\ set}$
$\mathit{Close}\ Bs\ B =$
$\mathit{Init} : x \in B \Longrightarrow x \in \mathit{Close}\ Bs\ B$
$\mathit{Predict} : [\![x \in \mathit{Close}\ Bs\ B;\ x' \in \mathit{Predict}\ x\ |Bs|]\!] \Longrightarrow x' \in \mathit{Close}\ Bs\ B$
$\mathit{Complete} : y \in \mathit{Close}\ Bs\ B \Longrightarrow \mathit{is_complete}\ y \Longrightarrow \mathit{from}\ y = |Bs| \longrightarrow x \in \mathit{Close}\ Bs\ B$
$\Longrightarrow \mathit{from}\ y < |Bs| \longrightarrow x \in Bs\ !\ \mathit{from}\ y \Longrightarrow \mathit{next_sym_Nt}\ x\ (\mathit{lhs}\ (\mathit{prod}\ y))$
$\Longrightarrow \mathit{mv_dot}\ x \in \mathit{Close}\ Bs\ B$

We then prove equivalence of the *bins* method using *Close* to the original Earley implementation.

Theorem 3. $i \leq |w| \Longrightarrow \mathit{bins}\ |w|\ !\ i = \mathcal{S}\ i$

Here the function $\mathcal{S}$ collects all items of the fully inductive Earley-set that are in the i-th set.

We are working towards a one pass closure algorithm, where we only need to consider every element once. This is possible if the items added by *predict* and *complete*, do not

depend on the items in the current set closure we are computing. This is the case for the predict operation, which only depends on the set of productions. For the complete operation it is only the case, if the complete items from-value is the index of a previous bin. This is equivalent to the CFG not having any productions of length zero. The handling of the complete operation for length zero productions it not straight-forward, so we require the grammar to be epsilon free.

Definition 3. *Eps_free* :: $('a \times 'b\ list)\ set \Rightarrow bool$

Eps_free $P = (\forall\, (lhs,\ rhs) \in P.\ rhs \neq [])$

This way we simplify the algorithm. We also give some options on how to deal with not epsilon free grammars in 3.4. So we create another closure under this assumption *Close1* which works the same as *Close*, but the complete operation is simplified, as we assume the grammar to be epsilon free:

Complete : $y \in$ *Close1* $Bs\ B \Longrightarrow$ *is_complete* $y \Longrightarrow x \in Bs\ !$ *from* y

$\Longrightarrow$ *next_sym_Nt* x (*lhs* (*prod* y)) $\Longrightarrow$ *mv_dot* $x \in$ *Close1* $Bs\ B$

We prove equivalence to *Close* under the assumption of epsilon free-ness.

Lemma 2. *Eps_free* $P \Longrightarrow$ *wf_bins1* $Bs \Longrightarrow$ *wf_bin1* $B\ |Bs|$

$\Longrightarrow$ *Close1* $Bs\ B =$ *Close* $Bs\ B$

Unless specified otherwise we will assume that the grammar is epsilon free from here on out. The next step towards executability is replacing the inductive closures with a one pass worklist algorithm.

3.3 Workset closure

We keep track of a pair of sets, the workset B and the accumulator C. The workset comprises items in the closure which have yet to be considered, while the accumulator keeps track of all previously considered items in the closure. So at any step $B \cup C$ are all items we know to be in the closure and it should hold that $B \cap C = \emptyset$. One step of this worklist algorithm then looks like this:

- choose an item from B to be considered $b \in B$
- compute the set of items D added by b under *predict* or *complete*
- move b from B to C and add D to B, while making sure that $B \cap C = \emptyset$ by the end

The last step we do as follows $(B \cup D - (C \cup \{b\}),\ C \cup \{b\})$. To formalize this approach we define a step relation that relates any two such set pairs that differ by one step.

Definition 4. *Close2* ::
$('n,\ 'a)$ *item set list*
$\Rightarrow ('n,\ 'a)$ *item set* $\times\ ('n,\ 'a)$ *item set*
$\Rightarrow ('n,\ 'a)$ *item set* $\times\ ('n,\ 'a)$ *item set* $\Rightarrow$ *bool*

Predict : $x \in B \Longrightarrow \neg$ *is_complete* x

$\Longrightarrow Bs \vdash (B,\ C) \rightarrow (B \cup \textsf{Predict}\ x\ |Bs| - (C \cup \{x\}),\ \{x\} \cup C)$

$Complete : x \in B \Longrightarrow \textsf{is_complete}\ x$
$\Longrightarrow Bs \vdash (B,\ C) \rightarrow (B \cup \textsf{Complete}\ Bs\ x - (C \cup \{x\}),\ C \cup \{x\})$

The closure is then given by the set C when repeated steps result in $B = \emptyset$. This will always be the case, as the set of valid Earley items is finite. More on this in 7. Formally this means we are looking for the accumulator C of a set pair $(\emptyset,\ C)$ that is reachable from $(B,\ \emptyset)$ under the step realtion. But since this formalisation is non-deterministic in the order that items are chosen from the workset we require the Hilbert choice operator for the formalisation to get a random but distinct set C if it exists. In reality this set is unique and always exists.

Definition 5. *close2* :: $('n,\ 'a)$ *item set list* $\Rightarrow$ $('n,\ 'a)$ *item set* $\Rightarrow$ $('n,\ 'a)$ *item set*
close2 $Bs\ B = (SOME\ C.\ Bs \vdash (B,\ \emptyset) \rightarrow* (\emptyset,\ C))$

We again prove equivalence of *close2* to *Close1*.

Lemma 3. $[\![\textsf{wf_bins1}\ Bs;\ \textsf{wf_bin1}\ B\ |Bs|]\!] \Longrightarrow \textsf{close2}\ Bs\ B = \textsf{Close1}\ Bs\ B$

For this proof we need two other lemmas. First that *close2* terminates and second that the result is equal to the set obtained by *Close1*. We will first focus on the set equality. The direction that *close2* is a subset of *Close1* is simple, by the fact that the result after one step is always a subset of the final result of *Close1*. The other direction is a bit trickier. We do a proof by induction on the items in *Close1* and have to check three cases. First the initial items in B are also in the result of *close2*, which is easily apparent as any item in B will at some point be moved to C. Second for the induction step we have to prove that under the assumption that an item $x \in \textsf{Close1}\ Bs\ B$ is also in C of *close2*, all items obtained by the operations *predict* or *complete* on x also end up in C. This proof boils down to the observation that if x ends up in C there must exist a step where it is picked from B and moved to C. In this step all of the items added to *Close1* by x are also added to B in the *Close2* step and therefore end up in C eventually.

For termination we define a well founded potential function which decreases with every step taken. It is a lexicographic ordering primarily on the number of well-formed items not in the worklist or accumulator, which decreases whenever an item is added through *predict* or *complete*, and secondarily on the size of the worklist, which decreases, if no new items are added, as one item is always moved to the accumulator.

After proving that the step relation always decreases this potential, we can prove termination of the algorithm and finish the equivalence proof of *close2* and *Close1*.

3.4 Epsilon treatment

- do epsilon treatment outside of this Algorithm (is there an isabelle implementation of epsilon removal (constructing L - ϵ))
- check which Nts could produce epsilon and change predict to also do epsilon completion for these Nts (would require changing underlying algorithm and doing a lot of proving)

- TODO check literature for different stratagies

4 Executable List based Version

We can now go on to implement a list based version. We first start off by implementing the four Earley operations using lists:

$\mathit{Init_L} :: (nat,\ int)\ item\ list$

$\mathit{Init_L} = \mathit{map}\ (\lambda p.\ \mathit{Item}\ p\ 0\ 0)\ (\mathit{filter}\ (\lambda p.\ \mathit{lhs}\ p = S)\ \mathit{ps})$

$\mathit{Predict_L} :: (nat,\ int)\ item \Rightarrow nat \Rightarrow (nat,\ int)\ item\ list$

$\mathit{Predict_L}\ x\ k = \mathit{map}\ (\lambda p.\ \mathit{Item}\ p\ 0\ k)\ (\mathit{filter}\ (\lambda p.\ \mathit{next_sym_Nt}\ x\ (\mathit{lhs}\ p))\ \mathit{ps})$

$\mathit{Complete_L} ::$

$(nat,\ int)\ item\ list\ list \Rightarrow (nat,\ int)\ item \Rightarrow (nat,\ int)\ item\ list$

$\mathit{Complete_L}\ Bs\ y =$

$\mathit{map}\ \mathit{mv_dot}\ (\mathit{filter}\ (\lambda b.\ \mathit{next_sym_Nt}\ b\ (\mathit{lhs}\ (\mathit{prod}\ y)))\ (Bs\ !\ \mathit{from}\ y))$

$\mathit{Scan_L} :: (nat,\ int)\ item\ list \Rightarrow nat \Rightarrow (nat,\ int)\ item\ list$

$\mathit{Scan_L}\ Bs\ k =$

$(\mathit{let}\ x = \mathit{Some}\ (w\ !\ k)\ \mathit{in}\ \mathit{map}\ \mathit{mv_dot}\ (\mathit{filter}\ (\lambda b.\ \mathit{next_symbol}\ b = x)\ Bs))$

We then go on to prove equivalence to the set based version defined earlier.

The closure is implemented using *while_option*.

5 Efficient Itemlist Datastructure for running time guarantees

Our aim is to prove the running time analysis given by Earley [1]. For this we require linear runtime of the *step_fun*, which is not given using standard list based set-functions, as the set difference operation takes $O(n^2)$ time when using lists. To fix this we follow Earley's idea to implement a $O(1)$ item lookup and insertion, which is enough to design an $O(n)$ set difference. The main idea is to group items in a set by their *from* value. For every value of *from* there are only a constant number of items. And the *from* are also in the range $[0, i]$ for closure of $\mathcal{S}_i$. This would allow for the storage of all items in an array of item lists where the list at index i has all items for *from* value i. Assuming constant access time for an array we can then implement *isin* and *insert* in $O(1)$.

- WorkList Datatype + why requirement for cubic runtime
- WorkList insert intracasies (set bahavior using upsize for ease of theoretical analysis, but isnt used in actual algorithm)
- List based functions
- correctness by equivalence to inductive set based WorkListAlgorithm
- WorkList Algorithm with help of while_option

6 Runingtime Analysis

- analysis with WorkList T_nth parameter (assumed to be constatnt for array)
- WorkList insert requires WorkList map_length bound, which has to be carried through proofs
- runtime of list functions
- parametized runingtime (cubic if T_nth is constant)

7 Space Analysis

8 Expansion to Parse Trees

- set version which does not require only one tree per Earley item
- parse items and well formedness characterisation
- list version for executability
- in many ways similar proof to first version

9 Runingtime Analysis of Expanded Algorithm

- same O runingtime as version without (different prefactors)

10 Try out examples

$xs\ !\ n$ is the n-th element

$Px \wedge Q\ 1\ ...\ n$

Lemma 4. $[\![i \leq k;\ k \leq |w|]\!] \Longrightarrow \mathit{set}\ (\mathit{bins_L}\ k\ !\ i) = \mathcal{S}\ i$

„

Bibliography

[1] Earley, J.: An efficient context-free parsing algorithm. Communications of the ACM **13**(2), 94–102 (Feb 1970). https://doi.org/10.1145/362007.362035